

II-初识卷积神经网络

- 一张RGB三通道彩色图片是如何被处理成可用于训练的数据？张量又是什么？

一张RGB三通道彩色图片本质是一个 $\text{高} \times \text{宽} \times 3$ 的像素矩阵，三通道分别对应红、绿、蓝，每个像素值是 0~255 的整数。

读取图片后，我们将其**缩放/裁剪，归一化/标准化**，最后转为**张量**。

张量本质是**多维数组**，优点是支持**自动求导**，方便计算梯度。

- 一个卷积神经网络分为哪几层？

层	作用
输入层	接收图片张量
卷积层	用卷积核提取局部特征（边缘、纹理等）
激活层	引入非线性（常用 ReLU）
池化层	降维、扩大感受野、增强平移不变性
批归一化	稳定训练、加速收敛
全连接层	把高层特征整合，映射到类别得分
输出层	softmax 输出各类别概率

Model

单层：卷积 $\rightarrow$ $BN \rightarrow ReLU \rightarrow$ 池化

该题使用一个简单的 3 层卷积 + 池化 + 全连接的 CNN ：

```
输入 3×224×224

-> conv1(3->32) + BN + ReLU + MaxPool -> 32×112×112

-> conv2(32->64) + BN + ReLU + MaxPool -> 64×56×56

-> conv3(64->128) + BN + ReLU + MaxPool-> 128×28×28

-> flatten(128*28*28=100352)

-> fc1(100352 -> 512) + ReLU + Dropout

-> fc2(512 -> num_classes)          -> 各类别得分(logits)
```

conv卷积

一个核（小矩阵）在输入图上滑动，与局部区域做"逐元素相乘再求和"，得到一个输出值，最终产生一张特征图。

数学卷积与神经网络卷积的区别：

数学上的卷积： $(f*g)[n] = \sum_m f[m] \cdot g[n-m]$ ，需要对卷积核做翻转再滑动相乘；

神经网络里的卷积：实际上没有翻转，是 $(f*g)[n] = \sum_m f[m] \cdot g[n+m]$ 。

卷积核：

一个小矩阵（如 3×3 ），每个核负责检测一种局部特征。一张图会用到多个卷积核，每个核输出一个通道的特征图。

特点：

卷积核初始是由随机数填充的，接着通过不断学习来调整数值。

作用：

一个卷积核就可以生成一张新图，如 `conv1(3->32)` 表示原有3通道，第一次卷积需输出32通道，于是我们使用32个 $3 \times 3 \times 3$ 的核对原图卷积就可以了。

步长：

卷积核每次滑动的像素距离。步长越大，输出尺寸越小、计算量越小。

填充：

在图片四周补一圈值（通常是 0），再拿去卷积。维持图的尺寸和充分利用边缘像素。

BN批归一化

把神经网络中间某一层的输出，重新拉回到「均值 0、方差 1」的标准分布，让训练更稳、更快。

解决问题：

训练过程中，前面几层的数值**每步都在更新**。这导致一个连锁反应：某一层的输入会受上层输出影响，导致变化累积。

它带来三个坏处：

1. **训练慢**：后面层老在适应新数值，收敛拖沓。
2. **对学习率敏感**：分布变化累积，稍微大一点的学习率就直接发散。
3. **容易梯度消失/爆炸**：值越传越大或越小。

解决方法：

在每一层之后进行**归一化**，把输出强行拉回均值 0、方差 1，切断数值漂移。这样每一层的输入分布都较为稳定。

注意：

并不是简单的**归一化**，还要通过 γ 和 β 两个**可学习的参数**进行放缩和平移。以避免因为强行归一到 0 均值 1 方差**可能会破坏特征**。

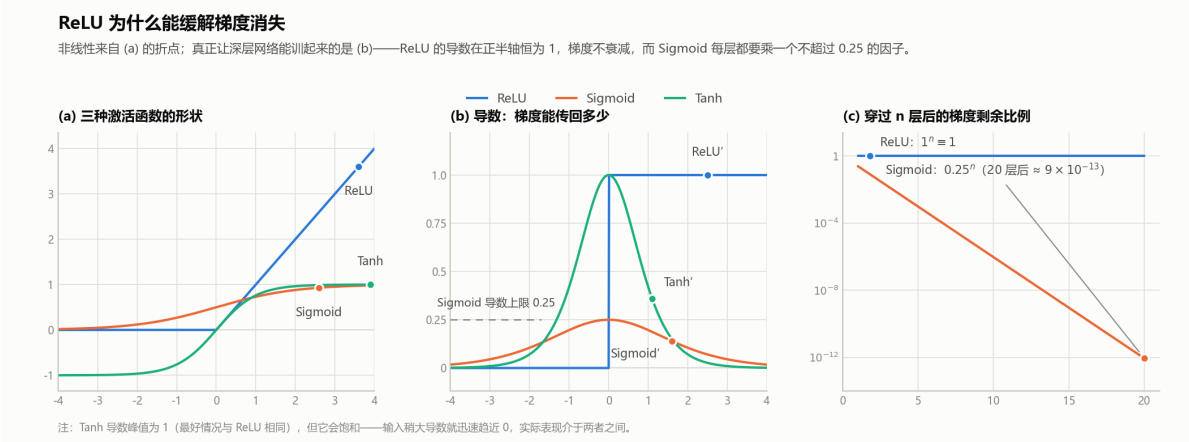
好处：

既能稳定分布，又不会束缚网络。

ReLU激活函数

$$\text{ReLU} = \max(0, x)$$

引入**非线性**。卷积、全连接本质上都是线性变换，若没有非线性激活，无论堆多少层都等价于一个线性变换，无法拟合复杂函数。



池化

用一个窗口在特征图上滑动，取窗口里的「一个代表值」，从而把图缩小。

常见池化类型：

类型	操作	特点
最大池化 Max Pooling	取窗口内最大值	保留最显著特征，对纹理/边缘敏感，最常用
平均池化 Average Pooling	取窗口内平均值	更平滑，保留整体背景信息
全局池化 Global Pooling	对整张特征图池化	常用于替代全连接层，减少参数

池化的作用：

1. **降维**：减小特征图尺寸，大幅减少后续参数数量和计算量。
2. **扩大感受野**：让更深层能看到更大范围的输入。
3. **增强平移**：局部位置的微小移动不影响结果。

感受野：

输出特征图上的一个单元对应**原始输入图像上的区域大小**。层越深、卷积核越大、步长越大，感受野越大。感受野大意味着特征图上能看到更大的上下文信息。

fc全连接

前面所有特征综合起来算一个加权和，每个输出都连着每个输入。

卷积负责**提取特征**（这一步在空间里，找边缘、纹理、花蕊），全连接负责**做决定**（把特征综合成类别判断）。

$$y = W \cdot x + b$$

- **x**：输入特征
- **W**：权重矩阵
- **y**：输出

Dropout随机失活

训练时随机清空一部分神经元（把它们输出设成 0），逼每个神经元都独立有用，从而防止过拟合。

核心代码

```
def __init__(self, num_classes=5):
    super(MyModel, self).__init__()
    self.conv1 = nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1)
    self.bn1 = nn.BatchNorm2d(32)
    self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
    self.bn2 = nn.BatchNorm2d(64)
    self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1)
    self.bn3 = nn.BatchNorm2d(128)
    self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
    self.fc1 = nn.Linear(128 * 28 * 28, 512)
    self.fc2 = nn.Linear(512, num_classes)
    self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.pool(F.relu(self.bn1(self.conv1(x))))
        x = self.pool(F.relu(self.bn2(self.conv2(x))))
        x = self.pool(F.relu(self.bn3(self.conv3(x))))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

Preprocess

对数据进行预处理。

要求：

1. 检测并删除**损坏、无法打开**的图片。
2. 按类别把数据集划分为 **训练集**(train)、**验证集**(val)、**测试集**(test)。

- 3. 数据增强（扩增样本）与统一尺寸。
- 4. 使用 DataLoader 和 DataSet 进行训练。

代码详解：

1)

```
sys.path.insert(0, str(Path(__file__).resolve().parent))
import config
```

统一使用 config 进行超参数配置，方便快速调整超参数与部分数据。

```
def is_valid_image(path: Path) -> bool:
    try:
        with Image.open(path) as img:
            img.verify()
        with Image.open(path) as img:
            img.load()
            if img.size[0] == 0 or img.size[1] == 0:
                return False
        return True
    except Exception:
        return False
```

verify：校验文件头与结构完整性。

load：解码图片，筛选损坏的图片。

最后筛选的空白（无像素点）的图片。

	verify()	load()
检查什么	文件头、结构	像素数据能否解码
效率	很快	慢（整张图进内存）
调用后对象状态	作废	正常可用

两个是互补的：verify() 抓「格式不对」，load() 抓「数据不全」。只用一个都漏。

2)

```
def _split_ratio(n, val_ratio, test_ratio):
    n_test = int(n * test_ratio)
    n_val = int(n * val_ratio)
    return n - n_test - n_val, n_val, n_test
```

按比例算出 (train, val, test) 三份数量。

```
def _copy(cls_dir: Path, paths, out_dir: Path, split: str):
    dst = out_dir / split / cls_dir.name
    dst.mkdir(parents=True, exist_ok=True)
    for p in paths:
        shutil.copy2(p, dst / p.name)
```

将一组照片复制至 `out_dir/split/类别名/` 下。

```
for p in cls.iterdir():
    total += 1
    if is_valid_image(p):
        valid.append(p)
    else:
        p.unlink()
        removed += 1
```

扫描**所有照片**并删除**无效照片**。

```
_copy(cls, valid[:n_train], out_dir, "train")
_copy(cls, valid[n_train:n_train + n_val], out_dir, "val")
_copy(cls, valid[n_train + n_val:], out_dir, "test")
```

按比例划分，保证每类的 `train/val/test` 比例一致（分层抽样）

3)

```
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]
```

这两个常量是 *ImageNet* 数据集上、按 R/G/B 三个通道分别统计出来的**均值**和**标准差**。

我们使用这两个常数来**归一化**。

```
return transforms.Compose([
    transforms.RandomResizedCrop(config.IMG_SIZE, scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2,
hue=0.1),
    transforms.ToTensor(),
    normalize,
])
```

增强训练集：随机裁剪缩放 + 翻转 + 旋转 + 颜色抖动，扩增样本容量

```
return transforms.Compose([
    transforms.Resize((config.IMG_SIZE, config.IMG_SIZE)),
    transforms.ToTensor(),
    normalize,
])
```

缩放+归一化。

4)

- *Dataset*: 数据集的抽象, 负责"存数据、取样本", 需实现。

`torchvision.datasets.ImageFolder` 就是现成的 *Dataset*, 能对图片进行排序标号。

- *DataLoader*: 对 *Dataset* 的封装, 负责**批量 (batch) 加载、打乱 (shuffle)、多进程读取 (num_workers)**, 训练时直接 `for images, labels in loader` 迭代即可。

使用 *DataLoader* 每次从 *Dataset* 中取出 *batch_size* 组数据, 训练集中数据需要随机取出。

注意: 此题的训练集需要**增强+缩放+归一化**, 而验证集与测试集不需要**增强**。

config

统一使用 config 进行超参数配置, 方便快速调整超参数与部分数据。

数据:

`IMG_SIZE = 224` 统一缩放到的尺寸 (CNN 输入大小)

`BATCH_SIZE = 32` 批次大小

`NUM_WORKERS = 2` 数据加载进程数 (高效利用 *gpu*)

`VAL_RATIO = 0.15` 验证集比例

`TEST_RATIO = 0.15` 测试集比例

`SEED = 214` 随机种子, 保证可复现

超参数:

在训练开始前由人设定的参数, **不通过训练学习**。

`LEARNING_RATE = 1e-3` 学习率

`EPOCHS = 20` 训练轮数

`WEIGHT_DECAY = 1e-4` 权重衰减 (L2 正则, 抑制过拟合)

总损失 = 原来的损失 + $\lambda/2 \cdot \sum w^2$, λ 就是权重衰减, 减低部分过大权重对 loss 的影响, 抑制过拟合。

权重小 → **输出对输入的微小变化不敏感** → **决策边界更平滑**。模型学的是"大致规律", 而不是"精确记住每个训练点"。

`STEP_SIZE = 7` 学习率衰减周期 (每多少个 epoch 衰减一次)

`GAMMA = 0.1` 学习率衰减因子

优化器:

根据梯度来更新模型参数的算法, 目标是让损失函数最小化。

常见类型:

- *SGD*: 随机梯度下降。
- *AdaGrad*: 自适应学习率, 适合稀疏数据。
- *RMSProp*: 改进 *AdaGrad*, 缓解学习率衰减过快。
- *Adam* (本题使用): 结合动量 + 自适应学习率, 收敛快、对超参数不敏感。

```
opt = torch.optim.Adam(model.parameters(), lr = 1e-3)
```

`model.parameters()`: 把模型里所有可学习的参数 (w 和 b) 交给优化器管理

$lr = 1e-3$: 学习率 = 0.001, 每次更新的步长。

```
pred = model(xb).squeeze(1)
loss = loss_fn(pred, yb)
opt.zero_grad()
loss.backward()
opt.step()
```

优化器的作用: 损失算出来、反向传播求出梯度后, 优化器根据梯度去更新参数, 让下一次预测更准。

`opt.zero_grad()`: 清空梯度。

`loss.backward()`: 从`loss`出发回溯, 求出每个参数的梯度。

`opt.step()`: 更新参数。

所以我们只需循环这几步就可以了。

V-模型的优化与微调

层数越深、性能就一定越好吗？

不是。 理论上层数越深表达能力越强, 但实际会遇到问题:

1. **梯度消失 / 爆炸**: 层太深梯度在反向传播时被连乘放大或缩小到消失。
2. **过拟合**: 参数过多、数据不够时, 出现过拟合。
3. **退化问题**: 并非过拟合——训练误差累积, 深层网络难以优化。
4. **计算/显存消耗大**。

模型训练好了该如何使用呢？难道每次使用前都需要重新训练一次？

训练一次即可, 之后直接用保存的最优权重, 无需重新训练。

```
if val_acc > best_val_acc:
    best_val_acc = val_acc
    torch.save(model.state_dict(), os.path.join(config.MODEL_DIR, "best.pth"))
```

保存准确率最高的权重。